String Manipulation

CSE 101 Section 03

Introduction to Algorithms and Computing

Fall 2023

© Royal Military College of Canada

The String data type

- Text is represented in programs by the *string* data type.

- A string is a sequence of characters enclosed within

  - quotation marks (") or “Hello”
  - apostrophes (‘) ‘Hello’

Creating a String

```
str1 = "Hello"
str2='spam'
print(str1, str2)
print(type(str1))
print(type(str2))
```

Output:

Hello spam
<class 'str'>
<class 'str'>

Getting a String as an input

- Getting a String as an input:

```
firstName = input("Please enter your name: ")
print("Hello", firstName)
```

Indexing

- We can access the individual characters in a string through *indexing*.

- The positions in a string are numbered from the left, starting with 0.

- The general form is <string>[<expr>], where the value of expr determines which character is selected from the string.

```
str = 'badaboom'
print(str[3])
```

Indexing

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

```
greet = "Hello Bob"
greet[0]
print(greet[0], greet[2], greet[4])
x = 8
print(greet[x - 2])
```

Indexing

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

- In a string of *n* characters, the last character is at position *n-1* since we start counting with 0.
- We can index from the right side using negative indexes.

```
greet[-1]      b
greet[-3]      B
```

Slicing

- Indexing returns a string containing a single character from a larger string.
- We can also access a contiguous sequence of characters, called a *substring*, through a process called *slicing*.
  - Same as for regular lists – recall Quiz 4

Slicing

- Slicing:
  <string>[<start>:<end>]
- start and end should both be integers

- The slice contains the substring beginning at position start and runs up to **but doesn't include** the position end.

Slicing

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

greet[0:3] → 'Hel'
greet[5:9] → ' Bob'
greet[:5] → 'Hello'
greet[5:] → ' Bob'
greet[:] → 'Hello Bob'

Reverse a String

To reverse a string using slicing, you use the syntax `string[::-1]`, where:

- string is the string to reverse.
- The first colon `:` signifies the start and end of the slice (left empty to include the whole string).
- The `-1` specifies the step, dictating that we move backward through the string by one character at a time.

```
original_string = "Hello"
reversed_string = original_string[::-1]
print(reversed_string)  # Outputs: "olleH"
```

Concatenation and Repetition

- Can we put two strings together into a longer string?

- *Concatenation* "glues" two strings together (+)

- *Repetition* builds up a string by multiple concatenations of a string with itself (*)

Concatenation and Repetition

```
"spam" + "eggs”

"Spam" + "And" + "Eggs”

3 * "spam”

"spam" * 5

(3 * "spam") + ("eggs" * 5)
```

len() and loops with strings

```
len("spam")

for ch in "Spam!":
    print (ch, end=" ")
```

The String data type

Operator	Meaning
+	Concatenation
*	Repetition
<string>[]	Indexing
<string>[:]	Slicing
len(<string>)	Length
for <var> in <string>	Iteration through characters

Simple String Processing

- Usernames on a computer system
  - First initial, first seven characters of last name

```
# get user's first and last names
first = input("Please enter your first name (all lowercase): ")
last = input("Please enter your last name (all lowercase): ")

# concatenate first initial with 7 chars of last name
uname = first[0] + last[:7]
```

Strings vs. Lists

Strings and lists in Python are both ordered *sequences* of elements.

```
# Concatenation
[1,2] + [3,4] # [1, 2, 3, 4]
# Repetition
[1,2] * 3 # [1, 2, 1, 2, 1, 2]
# Indexing and slicing
grades = ['A', 'B', 'C', 'D', 'F']
print(grades)
grades[0]
grades[2:4]
# len() to get length
len(grades)
```

Strings vs. Lists

But are different in several ways:

- Strings are always sequences of characters, but *lists* can be sequences of arbitrary values.
  - Lists can have numbers, strings, or both!

    myList = [1, "Spam ", 4, "U"]

- Lists are *mutable* but strings are *immutable*.
  - Lists in Python *can be* changed after they are created, meaning you can add, remove, or modify elements within a list.
  - Strings in Python are immutable, which means you cannot change their content once they are created.

Strings and Lists

- We can make a list of months:

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep",
"Oct", "Nov", "Dec"]
```

- months is list of strings.

Strings and Lists

```
def main():
    # months is a list used as a lookup table
    months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

    n = eval(input("Enter a month number (1-12): "))

    print("The month abbreviation is", months[n - 1] + ".")

    print(months[n-1][0])

main()
```

Operation	Description	Example
Concatenation (+)	Combining two strings to create a new one.	"Hello, " + "World!" yields "Hello, World!"
String Interpolation (f"{}")	Inserting variables or expressions into a string.	name = "Alice” f"Hello, {name}" yields "Hello, Alice"
Length (len())	Finding the number of characters in a string.	len("Python") returns 6
Indexing (str[index])	Accessing a specific character by its index.	"Python"[0] returns "P"
Slicing (str[start:end])	Extracting a substring from a string.	"Python"[1:4] returns "yth"
Splitting (str.split())	Dividing a string into a list of substrings.	"Hello, World".split(", ") returns ["Hello", "World"]
Stripping (str.strip())	Removing leading and trailing whitespace.	" Hello ".strip() returns "Hello"
Upper/Lower Case (str.upper(), str.lower())	Changing case of characters.	"Python".upper() returns "PYTHON"

Operation	Description	Example
find(substring)	Searches for a substring within the string and returns the lowest index where the substring is found. If not found, it returns -1.	"Hello, World".find("World") returns 7, "Hello, World".find("Python") returns -1
count(substring)	Counts the number of non-overlapping occurrences of a substring within the string.	"abracadabra".count("a") returns 5, "pythonpython".count("py") returns 2
Membership (in)	Checking if a substring exists in a string.	"Python" in "Hello, Python" returns True
Formatting (str.format())	Creating formatted strings with placeholders.	"Hello, {}".format("Alice") yields "Hello, Alice"
Joining (str.join())	Combining a list of strings into a single string.	", ".join(["Alice", "Bob"]) yields "Alice, Bob"
Replace (str.replace())	Replacing one substring with another.	"Hello, World".replace("World", "Python") returns "Hello, Python"

Exercise: countWords

- Write a Python function named countWords that will count the number of words in a string.
- The function should take a string and return the number of words in the string. The function should return 0 if an empty string is passed to the function.
- For the purposes of this function, assume words in the string are separated by one or more spaces.

```python
def countWords(input_string):
    # Remove any trailing spaces
    input_string = input_string.strip()
    if len(input_string) == 0:
        return 0
    # Initialize a word count
    word_count = 0
    # Iterate through the characters in the string
    for char in input_string:
        # If a space is encountered, increment the word count
        if char == ' ':
            word_count += 1
    return word_count

# Test cases
empty_string = ""
text = "This is a sample sentence     with several words."
word_count_empty = countWords(empty_string)  # Should return 0
word_count_text = countWords(text)  # Should return 7
print("Word count (empty string):", word_count_empty)
print("Word count (text):", word_count_text)
```

This solution has a few problems…
Let's figure them out

Improved countWords

- To improve countWords, further assume that words are sequences of characters that start with a letter. For example: 'Twolizards' and 'lizards2' are considered words but '2lizards' is not a word.
- Test your function for the following strings: 'Two lizards and 3 toads' and '20 lizards and 32 toads', and 'lizards2 and 32toads'.
- Use the function isalpha() to check is a character is a letter or a number.

```python
def countWords2(input_string):
    # Remove leading and trailing whitespace
    input_string = input_string.strip()

    # Check if the input string is empty
    if len(input_string) == 0:
        return 0
    else:
        # Check if the first character is a letter
        if input_string[0].isalpha():
            num = 1
        else:
            num = 0
        # Iterate through the characters in the string
        for c in range(1, len(input_string) - 1):
            if input_string[c] == ' ' and input_string[c + 1].isalpha():
                num += 1
    return num
```